

Trabajo Práctico N° 1

Problema 3

Implementar en python los siguientes algoritmos de ordenamiento:

- a) Ordenamiento burbuja
- b) Ordenamiento quicksort
- c) Ordenamiento por residuos (radix sort)

Corroborar que funcionen correctamente con listas de números aleatorios de cinco dígitos generados aleatoriamente (mínimamente de 500 números en adelante).

Medir los tiempos de ejecución de tales métodos con listas de tamaño entre 1 y 1000. Graficar en una misma figura los tiempos obtenidos. ¿Cuál es el orden de complejidad O de cada algoritmo? ¿Cómo lo justifica con un análisis a priori?

Comparar ahora con la función built-in de python **sorted**. ¿Cómo funciona sorted? Investigar y explicar brevemente.

ENTREGABLE DEL INFORME 1: Comparación de algoritmos de ordenamiento

En este trabajo se implementaron tres algoritmos de ordenamiento: Burbuja, Quicksort y Radix Sort, y se compararon sus tiempos de ejecución con la función incorporada sorted () de Python (basada en el algoritmo Timsort).

Análisis experimental

El objetivo fue ordenar listas de como mínimo 500 números de cinco dígitos generados aleatoriamente. Para esto generamos una función que produce 750 números aleatorios de cinco dígitos (750 es el valor elegido arbitrariamente para el experimento, que cumple con el rango pedido "listas de como mínimo 500 números") para realizar las pruebas. Luego verificamos que los algoritmos funcionan correctamente, midiendo sus tiempos de ejecución (también con la utilización de una función) para tamaños de listas entre 1 y 1000 elementos.

Posteriormente se graficaron todos los algoritmos en una misma figura. Y se hizo un segundo análisis omitiendo el ordenamiento burbuja en una segunda figura, ya que su crecimiento cuadrático distorsionaba la escala y no permitía distinguir correctamente las diferencias entre los métodos más eficientes. Las curvas están muy cerca entre sí o con mucho ruido (picos y valles), no se distingue tan claro el crecimiento teórico esperado (lineal, algorítmico, cuadrático, etc.).

Breve explicación de cada ordenamiento:

Ordenamiento burbuja:

- Realiza múltiples pasadas a lo largo de una lista, comparando e intercambiando los ítems adyacentes si no están en orden. En cada pasada a lo largo de la lista ubica el siguiente valor más grande en su lugar apropiado. En esencia, cada ítem “burbujea” hasta el lugar al que pertenece. Este proceso se repite hasta que la lista queda ordenada.

Ordenamiento quicksort:

- Dividir y conquistar. Selecciona un pivote, en este caso es el primer elemento y particiona la lista en dos sublistas: elementos menores al pivote a la izquierda y mayores a la derecha. El procedimiento se aplica recursivamente a las sublistas hasta obtener la lista ordenada.

Ordenamiento radix sort:

- Funciona examinando los números dígito por dígito, comenzando por el dígito menos significativo (unidades). Los números se distribuyen en "cajitas" (cubetas) según el valor de dicho dígito. Las cajitas se recolectan en orden, y el proceso se repite con el siguiente dígito (decenas, centenas, etc.), hasta que todos los dígitos han sido procesados.

Ordenamiento sorted:

- Es la función de ordenamiento incorporada en Python. Devuelve una nueva lista ordenada a partir de cualquier iterable de entrada, sin modificar el original. Es un algoritmo híbrido que combina Merge Sort e Insertion Sort, conocido por su rendimiento, estabilidad y robustez.

Orden de complejidad de cada algoritmo a priori (Ver Anexo):

- Ordenamiento burbuja: El mejor de los casos es $O(n)$ (comportamiento lineal) cuando la lista ya está ordenada y el algoritmo detecta que no se realizaron intercambios. En el caso promedio y peor caso es $O(n^2)$ (comportamiento cuadrático) ya que compara e intercambia elementos adyacentes en cada pasada. Es el más ineficiente para listas grandes.

- Quicksort: en el mejor de los casos, la lista ya está ordenada, no se realizan cambios; la complejidad es $O(n \log n)$ (comportamiento logarítmico), el pivote divide la lista equilibradamente. En el caso promedio el pivote hace algunas divisiones equilibradas, la complejidad es $O(n \log n)$. En el peor de los casos, cada comparación causará un intercambio $O(n^2)$, el pivote siempre es el mínimo o máximo elemento (división muy desbalanceada). En la práctica es uno de los métodos más rápidos, si los puntos de división no están cerca de la mitad de la lista. Este ordenamiento no requiere espacio adicional.

- Radix sort: tiene un comportamiento bastante particular, porque no depende de comparaciones entre elementos, sino del número de dígitos a recorrer. Por eso su tiempo de ejecución no cambia mucho entre el mejor, el promedio y el peor caso, siempre que los números tengan la misma cantidad de dígitos. En el mejor de los casos, cuando los números tienen pocos dígitos (k pequeño) y el rango de valores es acotado la complejidad es $O(k \cdot n)$. En la práctica si todos tienen la misma cantidad de dígitos, se comporta como $O(n)$. En el caso promedio también es $O(k \cdot n)$, porque el algoritmo siempre recorre todos los dígitos de

todos los números. Incluso en el peor de los casos la complejidad sigue siendo $O(k \cdot n)$, porque el número de pasadas por dígitos no cambia. Solo podría empeorar si los números tienen muchísimos dígitos o si las operaciones por dígito son muy lentas. Para enteros de tamaño fijo se aproxima a $O(n)$, y lo hace más eficiente.

- Sorted: combina Merge Sort e Insertion Sort, en el mejor de los casos la lista esta parcialmente ordenada o completamente ordenada y Timsort detecta subsecuencias ya ordenadas y las aprovecha, por lo que apenas hace comparaciones, la complejidad es $O(n)$. En el promedio es $O(n \log n)$, la lista tiene cierto grado de desorden, y Timsort combina y fusiona esas subsecuencias de manera eficiente y en el peor de los casos también es $O(n \log n)$, incluso en el peor escenario (lista completamente desordenada o inversamente ordenada), el algoritmo mantiene la eficiencia de una Merge Sort optimizado. No hay degradación a $O(n^2)$ porque Timsort nunca se desbalancea. Es un algoritmo híbrido (mezcla de Merge Sort e Insertion Sort), estable y muy optimizado en Python.

Ordenamientos	Casos	Mejor	Promedio	Peor
Burbuja		$O(n)$	$O(n^2)$	$O(n^2)$
Quicksort		$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Radix sort		$O(k \cdot n)$	$O(k \cdot n)$	$O(k \cdot n)$
Sorted		$O(n)$	$O(n \log n)$	$O(n \log n)$

Resultados obtenidos (ver Figura A):

- Burbuja: el más lento, con crecimiento cuadrático evidente.
- Quicksort, Radix Sort y Sorted: mucho más rápidos, con comportamiento lineales, son rectas en este rango de tamaños y en comparativa con el ordenamiento burbuja.

Resultados obtenidos (ver Figura B):

- Burbuja: se omite este algoritmo para apreciar el comportamiento de los otros ordenamientos.
- Quicksort: crece de manera casi lineal – logarítmica, con pequeñas fluctuaciones. Los picos pueden deberse a la elección del pivote, porque si divide mal alguna iteración, la lista tarda un poco más. Coincide la complejidad con el mejor caso o el promedio $O(n \log n)$.
- Radix Sort: muestra un crecimiento y comportamiento muy similar al de Quicksort, casi lineal pero más suave. Depende de recorrer los dígitos (no de comparaciones), por eso tiene una pendiente constante y un tiempo estable. La complejidad es $O(k \cdot n)$.
- Sorted: Es el más rápido de todos, casi pegado al eje x. Lo que confirma su eficiencia y optimización. Coincide con peor caso y el promedio $O(n \log n)$.

Conclusión

Los resultados coinciden con el análisis teórico: algoritmos cuadráticos como Burbuja se vuelven inviables al crecer el tamaño de la entrada, mientras que algoritmos es $O(n \log n)$, o casi lineales como Quicksort, Radix y Timsort mantienen tiempos muy bajos. En Python, la mejor opción práctica siempre es sorted (), por ser optimizado, estable y robusto.



Figura A: Gráfica del tiempo de ejecución de todos los ordenamientos, para listas de tamaño entre 1 y 1000

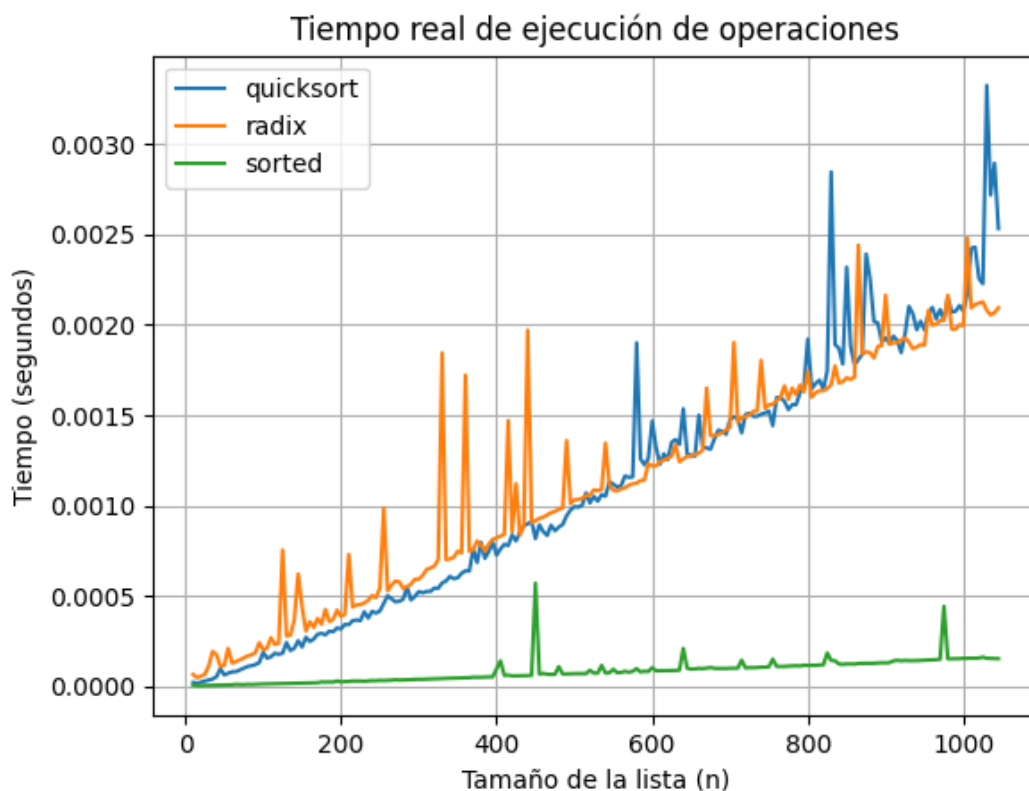
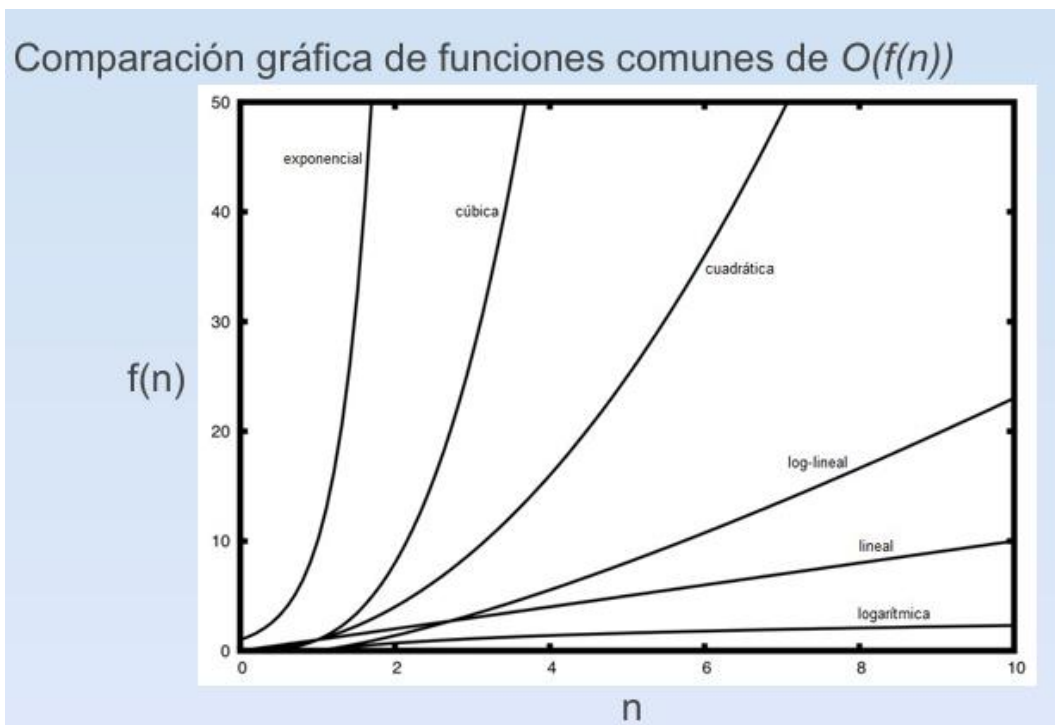


Figura B: Gráfica del tiempo de ejecución de los algoritmos más eficientes (quicksort, radix y sorted) usando tamaños de entre 1 y 1000. Se omitió el ordenamiento burbuja porque su crecimiento cuadrático distorsionaba la escala, dificultando observar las diferencias entre los demás algoritmos. Los picos irregulares corresponden a variaciones en el procesamiento, pero no alteran el comportamiento general esperado.

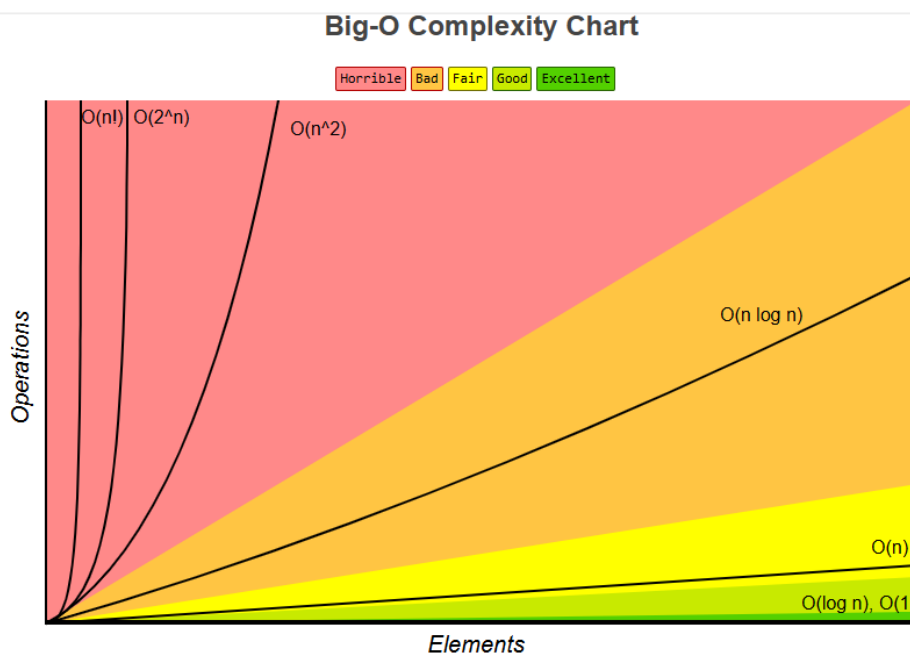
ANEXO

Gráficas 1 y 2 son utilizadas para comparar lo obtenido en Visual Studio Code con lo teórico



Gráfica 1: Comparación gráfica de funciones comunes de $O(f(n))$.

<https://runestone.academy/ns/books/published/pythoned/AlgorithmAnalysis/NotacionOGrande.html>



Gráfica 2: Comparaciones de las distintas complejidades

<https://www.freecodecamp.org/espanol/news/hoja-de-trucos-big-o/>